

Agente QA con Inteligencia Artificial

Documentación técnica de arquitectura y funcionalidades implementadas

Este documento describe la arquitectura, los módulos implementados y las capacidades actuales de un agente de QA automatizado, desarrollado en Java, que combina crawling web, análisis de páginas, generación de casos de prueba, generación de código Playwright real con Page Object Model, detección de secuencias de interacción, escaneo de seguridad básico e integración con un modelo de IA para responder preguntas en lenguaje natural sobre el análisis realizado.

Stack tecnológico

Lenguaje	Java
Automatización de navegador	Microsoft Playwright (Java)
Framework de testing	TestNG
Servidor web embebido	com.sun.net.httpserver.HttpServer
IA conversacional	Integración vía API (OpenRouter)
Empaquetado	Maven, Dockerfile, jar ejecutable
Interfaz	Chat web servido en localhost:8080 (HTML/CSS/JS embebido)

Arquitectura general

El agente sigue un flujo secuencial orquestado desde **AgentServer** (punto de entrada): navega al sitio objetivo, analiza la página, recorre el sitio completo, genera casos de prueba y código ejecutable, corre los tests contra la página real, escanea seguridad básica, y finalmente expone los resultados a través de un servidor web con interfaz de chat.

Paquetes y responsabilidades

- **agent** — AgentServer: orquesta todo el flujo y expone la interfaz web/chat.
- **analyzer** — PageAnalyzer, PageInfo, InputInfo, ButtonInfo: inspección estructural y de comportamiento de cada página.
- **crawler** — WebCrawler: recorrido multi-página del sitio (BFS, mismo dominio).
- **generator** — TestCaseGenerator, PageObjectGenerator, TestClassGenerator: generación de casos de prueba y código fuente Java/Playwright.
- **writer** — JavaTestWriter: persistencia en disco de los archivos .java generados, con creación automática de carpetas.
- **execution** — PlaywrightTestExecutor: ejecución real de acciones sobre la página analizada.
- **security** — XssScanner: detección de reflejo de XSS en inputs.

- **util** — `HtmlReportWriter`: generación del reporte HTML consolidado.
- **ai** — `AIAnalyzer`: integración con modelo de IA para responder preguntas sobre el análisis.

Flujo de ejecución

- 1 Se navega a la URL objetivo con Playwright (Chromium, headless).
- 2 **PageAnalyzer** inspecciona la página: inputs (con estrategia de identificación en cascada: name → id → placeholder → type), botones (texto, id, aria-label, title), formularios, links e imágenes.
- 3 El agente **prueba activamente cada botón** para detectar si revela campos ocultos (ver sección de Detección de secuencias de UI).
- 4 **WebCrawler** recorre hasta 50 páginas del mismo dominio, analizando cada una con la misma lógica.
- 5 **TestCaseGenerator** produce una descripción textual de casos de prueba (happy path, negative, boundary, interacción por botón).
- 6 **PageObjectGenerator** y **TestClassGenerator** generan código Java real: una clase Page Object y una clase de test TestNG completa, listas para compilar y ejecutar.
- 7 **PlaywrightTestExecutor** ejecuta acciones reales sobre la página (fill, click) y registra resultados PASS/FAIL.
- 8 **XssScanner** prueba reflejo de XSS en los inputs detectados.
- 9 **HtmlReportWriter** consolida todo en un reporte HTML navegable.
- 10 Se levanta un servidor HTTP con una interfaz de chat donde un modelo de IA responde preguntas sobre el análisis, usando todo el contexto recolectado como prompt.

Módulos implementados en detalle

1. Crawling multi-página

COMPLETADO

WebCrawler recorre el sitio a partir de la URL inicial, siguiendo enlaces del mismo dominio hasta un límite configurable (50 páginas en las pruebas realizadas).

- Análisis estructural por página: inputs, botones, forms, links, imágenes.
- Consolidación de totales del sitio completo en el reporte final.
- Probado end-to-end contra un sitio real (gub.uy) con resultado exitoso en 50/50 páginas.

2. Análisis estructural de página (PageAnalyzer)

COMPLETADO

Extrae de cada página los elementos interactivos relevantes para testing, con estrategias de identificación robustas y saneamiento de texto.

- Inputs: identificación en cascada name → id → placeholder → type, con fallback final si no hay ningún atributo útil.
- Botones: texto visible saneado (colapso de espacios/saltos de línea), más atributos id, aria-label y title capturados para selectores robustos.
- Cálculo de visibilidad real de cada input (page.isVisible()) para detectar campos ocultos.
- Conteo de formularios, links e imágenes por página.

3. Generación de casos de prueba (TestCaseGenerator)

COMPLETADO

Genera una descripción textual estructurada de casos de prueba a partir del análisis de la página.

- Caso happy path: completar todos los campos válidos y enviar.
- Caso negative: envío con campos vacíos.
- Casos boundary/negative por cada input individual.
- Un caso de interacción por cada botón detectado.

4. Generación de Page Objects (PageObjectGenerator)

COMPLETADO

Genera código Java real siguiendo el patrón Page Object Model, con selectores Playwright robustos y en cascada según la señal disponible de cada elemento.

- Un Locator y un método fillX(value) por cada input detectado, usando el selector más confiable disponible (name, id o placeholder).
- Un Locator y un método clickX() por cada botón, con selector en cascada: id → aria-label → title → getByRole(BUTTON, nombre accesible) como fallback final.
- Escapado de comillas y caracteres especiales en todo texto insertado dentro de strings Java generados, para evitar código roto por contenido real del sitio.
- Persistencia automática vía JavaTestWriter en src/main/java/generator/pageobjects/.

5. Generación de tests TestNG (TestClassGenerator)

COMPLETADO

Genera una clase de test TestNG completa y ejecutable que usa la Page Object correspondiente.

- Setup con navegación real a la URL analizada antes de cada test (page.navigate con espera de DOMCONTENTLOADED).
- Test de happy path, test de campos vacíos, y un test de boundary/negative por cada input.
- Respeta secuencias de UI: si un input requiere abrir un botón antes de ser usable, el test generado clickea ese botón primero automáticamente.

- Persistencia automática en `src/test/java/generator/tests/`.

6. Detección de secuencias de UI

COMPLETADO

Capacidad diferenciadora: el agente detecta cuándo un campo está oculto hasta que se ejecuta una acción previa (por ejemplo, un buscador que se despliega al hacer clic en un ícono), y genera los tests respetando esa secuencia real, sin intervención manual.

- Si existen inputs no visibles, el agente prueba cada botón de la página (click con timeout corto, tolerante a fallos).
- Tras cada click, revisa qué inputs pasaron a estar visibles y registra la relación input → botón que lo revela.
- Intenta cerrar el elemento revelado (muchos son toggles) para no dejar la página en un estado inconsistente antes de continuar probando otros botones.
- Validado en producción contra gub.uy: detectó correctamente que el campo de búsqueda requiere clic previo en 'Abrir buscador y Buscar'.

7. Ejecución real de pruebas (PlaywrightTestExecutor)

COMPLETADO

Ejecuta acciones reales (fill, click, visibilidad) contra la página analizada y registra resultados PASS/FAIL concretos, no simulados.

- Resultados incorporados al reporte HTML y al contexto que consume el chat con IA.

8. Seguridad básica: escaneo XSS reflejado (XssScanner)

COMPLETADO

Prueba los inputs detectados con payloads de XSS y verifica si se reflejan sin sanitizar en la respuesta de la página.

9. Reporte HTML consolidado (HtmlReportWriter)

COMPLETADO

Genera un archivo `reporte_qa.html` con resumen de la página, casos generados, resultados de ejecución y hallazgos de seguridad, servido también vía `/reporte` en el servidor embebido.

10. Interfaz conversacional con IA (AIAnalyzer + AgentServer)

COMPLETADO

Servidor HTTP embebido con una interfaz de chat donde el usuario puede preguntar en lenguaje natural sobre el análisis realizado.

- El contexto completo del análisis (estructura, casos generados, resultados reales, hallazgos de seguridad, resumen del crawling) se inyecta como prompt.
- Renderizado de respuestas en Markdown en el frontend.

11. Empaquetado y despliegue

COMPLETADO

El proyecto cuenta con Dockerfile para ejecución en contenedor y build de jar ejecutable vía Maven.

Roadmap: próximos módulos planificados

Los siguientes módulos están definidos como siguientes pasos del proyecto, priorizados por impacto en un perfil de QA moderno:

- **API Testing** — detección de endpoints y generación de pruebas GET/POST/PUT/DELETE con validación de status codes, headers, schema, autenticación y tiempo de respuesta.
- **Performance Testing con k6** — generación de scripts de carga con umbrales de response time, throughput y error rate.
- **Detección inteligente de bugs** — reglas de negocio (validaciones débiles, respuestas 200 con operaciones fallidas, etc.).
- **Generación automática de Bug Reports** — formato estructurado (ID, severidad, prioridad, pasos, evidencia).
- **Evidencia enriquecida** — screenshots, HTML, consola y red asociados a cada fallo detectado.
- **Accessibility Testing** — ARIA, labels, navegación por teclado, contraste, alt text.
- **Cross-browser testing** — ejecución en Chromium, Firefox y WebKit.
- **CI/CD** — integración con GitHub Actions / Jenkins / Azure DevOps.
- **Risk-Based Testing Engine** — priorización automática de esfuerzo de testing según criticidad de cada página/funcionalidad.